2026-11-04 로그인 폼의 비밀번호 표시와 저장된 아이디

- 날짜: 2026-11-04
- 앱: `lgcns-fe-next`
- 패키지: `next` 16.4.3, `react` 19.3.0, `next-auth` ^4.24.15 (`lgcns-fe-next/package.json`)
- 기준: 이 문서의 "현재 코드"는 지금 디스크의 파일이다. 저장소에 없는 동작은 있는 것처럼 쓰지 않는다.

이번 로그인 폼 변경은 두 가지다. 비밀번호 칸에 보기/숨기기 버튼을 켰고, 저장된 아이디를 `useEffect` 안의 `setState`로 넣던 코드를 `useSyncExternalStore`로 바꿨다. 비밀번호 문자열 자체는 `localStorage`에 넣지 않는다.

1. 파일이 언제 실행되는가

이 흐름의 소스에는 로그인 폼을 불러오는 `<script>`가 없다. Next가 모듈 그래프로 묶는다.

파일	선언	실행 시점	HTML <script>와의 차이
`lgcns-fe-next/app/(auth)/login/page.tsx`	`export default function Login()`. `"use client"`가 없다	서버가 로그인 페이지 HTML을 만들 때 실행된다. 자식은 `<Suspense fallback={null}>` 안의 `LoginSection`이다	브라우저용 스크립트 태그가 아니다. `async`, `defer`, `type="module"`로 이 파일을 직접 붙이지 않는다
`lgcns-fe-next/component/templates/LoginSection.tsx`	첫 줄 `"use client"`. `export default function LoginSection()`	서버가 최초 HTML을 만들고, 브라우저가 그 HTML을 hydrate할 때 다시 실행된다. `useSearchParams()`로 `callbackUrl`을 읽는다	`"use client"`는 `<script async>`도 `<script defer>`도 아니다. 이 파일과 그것이 import하는 클라이언트 모듈을 Next 번들에 넣으라는 지시문이다
`lgcns-fe-next/component/organisms/LoginForm.tsx`	첫 줄 `"use client"`. `export default function LoginForm({ callbackUrl })`	`LoginSection`이 렌더할 때 같이 렌더된다. 훅은 그 렌더 중에 호출된다. `localStorage` 쓰기는 폼 `submit` 때다	페이지 HTML에 따로 적힌 모듈 스크립트가 아니다. 부모 클라이언트 컴포넌트의 모듈 그래프에 포함된다
`lgcns-fe-next/component/molecules/FormField.tsx`	첫 줄 `'use client'`. `export default function FormField(...)`	`LoginForm`이 `<FormField>`를 그릴 때마다 실행된다. 비밀번호 표시 여부 `isVisible`은 이 컴포넌트의 `useState`다	로그인 폼이 이 파일을 import한다. 단독 `<script type="module">`로 로드되지 않는다
`lgcns-fe-next/components/ui/input.tsx`	`function Input` 후 `export { Input }`	`FormField`가 렌더할 때 호출된다. `type`을 `@base-ui/react/input`의 `InputPrimitive`에 넘긴다	named export다. `import Input from "..."`가 아니라 `import { Input } from "@/components/ui/input"`다
`lgcns-fe-next/types/initial/loginInitial.ts`	`const initialValues` 후 `export default initialValues`	`LoginForm`이 이 모듈을 import하는 순간 평가된다. 비밀번호 state의 처음 값 `""`만 여기서 온다	default export다. 아이디의 처음 화면 값은 이 객체가 아니라 아래 스냅샷이다
`lgcns-fe-next/schema/loginSchema.ts`	`const loginSchema`와 `function loginFieldMessage`. 파일 끝에서 `export { loginFieldMessage }`와 `export`	클라이언트 컴포넌트가 import하므로 브라우저 번들에서 `safeParse`가 돈다. 렌더마다 `loginSchema.safeParse(payload)`, 입력 `change`마다 `loginFieldMessage`	서버 액션이 아니다. 제출 전에 빈 문자열만 거른다

파일	선언	실행 시점	HTML <script>와의 차이
	default loginSchema		
lgcns-fe-next/types/request/RequestType.ts	interface UserPayloadType 후 export type { UserPayloadType }	타입만 export한다. 런타임 값이 없다	컴파일 후 지워진다. 스크립트 로딩과 무관하다

HTML에서 async는 스크립트를 받는 즉시 실행하고 파싱을 막지 않으며 순서를 보장하지 않는다. defer는 문서를 파싱한 뒤 DOMContentLoaded 전에 적힌 순서대로 실행한다. type="module"은 기본적으로 지연 실행되는 ES 모듈이고 import/export를 쓴다. 이 로그인 폼은 그 속성으로 붙지 않는다. Next가 "use client" 모듈을 클라이언트 번들로 만들고, 서버가 만든 HTML을 브라우저가 hydrate한 뒤에 훅이 돈다.

참고: HTML script 요소, MDN script, Next.js use client.

2. 비밀번호 보기와 숨기기

LoginForm의 비밀번호 FormField는 showPasswordToggle을 넘긴다. JSX에서 값 없는 showPasswordToggle은 showPasswordToggle={true}다. FormField의 매개변수 기본값도 true다. 토글이 그려지는 조건은 그 값이 참이고 type === "password"인 경우다.

표시 상태는 FormField 안의 useState(false)다. 컴포넌트 함수 본문에서 매 렌더 계산한다.

```
const showToggle = showPasswordToggle && type === "password";
const inputType = showToggle && isVisible ? "text" : type;
```

isVisible이 거짓이면 input의 type은 "password"다. 참이면 "text"다. 버튼 클릭은 setIsVisible((current) => !current)만 한다. 비밀번호 문자열은 바뀌지 않는다. LoginForm의 password state가 그 문자열을 들고, value={payload.password}로 넘긴다.

HTML 명세의 Password state는 이렇게 말한다. type이 password인 input은 한 줄 텍스트 편집 컨트롤이고, 사용자 에이전트는 사용자 외의 사람이 값을 볼 수 없도록 값을 가려야 한다. 줄바꿈 문자 U+000A, U+000D는 넣지 못한다. type을 "text"로 바꾸면 그 가리기 규칙이 빠진 한 줄 텍스트 상태가 된다. 브라우저의 눈 아이콘이 아니라, 이 코드가 type 속성을 바꿔 가리기를 끈다.

버튼은 type="button"이다. HTML에서 form 안의 button은 type을 생략하면 제출 버튼이다. 제출로 쓰면 눈 아이콘 클릭이 로그인 요청이 된다. type="button"은 기본 동작이 없다. onClick만 돈다.

접근 이름은 aria-label이다. 가려진 상태면 "비밀번호 보기", 보이는 상태면 "비밀번호 숨기기". aria-pressed={isVisible}라서 눌림 상태가 참/거짓으로 노출된다. 아이콘(Eye, EyeOff)에는 접근 이름이 없고, 버튼 이름이 그 역할을 한다.

로그인 폼은 isClearable을 넘기지 않는다. FormField의 지우기 버튼 조건 isClearable && currentValue는 여기서 거짓이다. 로그인 비밀번호 칸에는 지우기 버튼이 없다.

autoComplete는 아이디 "username", 비밀번호 "current-password"다. 명세의 autofill 힌트다. 저장소 코드가 브라우저 비밀번호 관리자를 따로 호출하지는 않는다.

참고: HTML Password state, MDN input password, HTML button type, MDN aria-pressed, React useState.

3. 저장된 아이디를 렌더 중에 읽기

키는 모듈 스코프 상수다.

```
const SAVED_USER_ID_KEY = "lgcns-saved-user-id";
```

`window.localStorage`는 오리진(스킴, 호스트, 포트)마다 하나인 `Storage`다. 세션이 끝나도 남는다. 키와 값은 UTF-16 문자열이다. `getItem`은 없으면 `null`을 반환한다. 이 코드는 `?? ""`로 빈 문자열을 만든다. `setItem`은 있으면 덮어쓰고, `removeItem`은 그 키만 지운다.

`localStorage`는 React state가 아니다. 브라우저가 들고 있는 값이라서 렌더 중에 `useState`의 초깃값으로 서버와 클라이언트가 다르게 읽으면 hydration이 어긋난다. 서버에는 `window`가 없다. React가 이 경우를 위해 둔 훅이 `useSyncExternalStore`다.

```
const savedUserId = useSyncExternalStore(
  subscribeSavedUserId,
  readSavedUserId,
  readSavedUserIdOnServer,
);
```

세 함수는 컴포넌트 밖에 선언돼 있다. React 문서는 `subscribe`를 컴포넌트 안에 두면 렌더마다 함수가 달라져서 구독을 다시 건다고 한다. 파일 스코프 함수는 그 재생성을 피한다.

함수	선언	하는 일
`subscribeSavedUserId`	function, export 아님	콜백을 등록하지 않고 `() => {}`를 반환한다
`readSavedUserId`	function, export 아님	브라우저에서 `localStorage.getItem(SAVED_USER_ID_KEY) ?? ""`
`readSavedUserIdOnServer`	function, export 아님	항상 `""`

`getServerSnapshot`은 서버에서 HTML을 만들 때와, 클라이언트가 그 HTML을 hydrate할 때 쓰인다. 둘 다 `""`이므로 첫 HTML의 아이디 칸은 비어 있다. hydrate가 끝난 뒤 React는 `getSnapshot`을 부른다. 저장된 문자열이 `""`와 `Object.is`로 다르면 그 문자열로 다시 렌더한다.

`subscribeSavedUserId`는 인자로 받은 콜백을 `storage` 이벤트에 넣지 않는다. React 문서의 `subscribe`는 스토어가 바뀌면 그 콜백을 호출해야 한다. 이 파일은 그 호출이 없다. 그래서 다른 탭이 같은 키를 바꿔도 이 폼은 그 변경만으로 다시 그리지 않는다. 처음 클라이언트 스냅샷을 읽는 것과, 이후 변경을 구독하는 것은 이 코드에서 따로다.

`getSnapshot`이 매 호출마다 새 객체를 반환하면 React는 "The result of getSnapshot should be cached"로 무한 렌더를 막는다. 이 함수의 반환은 문자열이다. 같은 문자열이 `Object.is`로 같으면 렌더를 반복하지 않는다.

참고: React useSyncExternalStore, React You Might Not Need an Effect, MDN localStorage, MDN Storage.getItem, ECMA-262 Object.is.

4. 화면 값과 사용자가 고친 값

저장된 문자열을 `useState`의 초깃값으로 복사하지 않는다. 렌더 중에 둘을 합친다.

```
const [userId, setUserId] = useState<string | null>(null);
const [password, setPassword] = useState(initialValues.password);
const [rememberChoice, setRememberChoice] = useState<boolean | null>(null);
const payload = {
  userId: userId ?? savedUserId,
  password,
};
const rememberId = rememberChoice ?? savedUserId.length > 0;
```

`??`는 Nullish coalescing이다. 왼쪽이 `null` 또는 `undefined`일 때만 오른쪽을 쓴다. `""`는 그 대상이 아니다.

- 사용자가 아이디를 아직 안 고쳤으면 `userId`는 `null`이다. 칸에는 `savedUserId`가 나온다.
- 사용자가 칸을 비우면 `setUserId("")`가 된다. `"" ?? savedUserId`는 `""`다. 저장된 아이디가 다시 채워지지 않는다.

- `rememberChoice`가 `null`이면 저장된 아이디가 한 글자 이상일 때 체크박스가 켜진다.
- 체크박스를 누르면 `setRememberChoice(event.target.checked)`라서 그 다음부터는 저장값 길이와 상관없이 사용자의 불리언이 이긴다.

체크박스는 `checked={rememberId}`인 제어 컴포넌트다. HTML checkbox는 요소의 checkedness가 참이면 선택, 거짓이면 비선택인 두 상태 컨트롤이다. 이 입력의 `onChange`가 React state를 갱신하고, 다음 렌더의 `checked`가 화면을 맞춘다.

제출 핸들러는 `event.preventDefault()`로 브라우저의 기본 폼 전송을 막는다. `loginSchema`는 `userId`, `password` 모두 `min(1)`이다. 실패하면 `loginFieldMessage`로 필드 오류만 넣고 `return`한다. 이때 `localStorage`는 그대로다.

통과하면 그때 저장한다.

```
if (rememberId) window.localStorage.setItem(SAVED_USER_ID_KEY, payload.userId);
else window.localStorage.removeItem(SAVED_USER_ID_KEY);
```

그 다음 `startTransition` 안에서 `signIn("credentials", { email: payload.userId, password: payload.password, redirect: true, callbackUrl })`를 호출한다. `useTransition`의 `isPending`이 참인 동안 버튼 문구는 `"로그인 중..."`이고 `aria-busy`가 참이다. 이 핸들러는 `signIn`의 반환값을 검사하지 않는다.

참고: MDN Nullish coalescing, ECMA-262 ??, HTML checkbox, React useTransition.

5. 이전 effect가 막힌 이유

이전 코드는 마운트 effect에서 `localStorage`를 읽고 `setRememberId(true)`, `setPayload(...)`를 바로 호출했다. `react-hooks/set-state-in-effect`는 effect 본문에서 `setState`를 동기로 호출하는 이 패턴을 오류로 본다. React 문서는 effect를 React 밖의 시스템과 맞추는 일에 쓰고, 렌더 중에 계산할 수 있는 값을 effect로 복사하지 말라고 한다.

지금 코드에서 저장된 아이디와 체크 여부는 렌더 중 계산이다. `localStorage`를 읽는 쪽만 외부 스토어 훅이 담당한다. 사용자가 친 아이디, 비밀번호, 체크 선택, 필드 오류는 `useState`다.